

Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ
по лабораторной работе № 2
на тему
«СИСТЕМА УПРАВЛЕНИЯ ВЕРСИЯМИ»

Студент:

М.А. Тукайло

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить порядок работы с Git, освоить основные операции с репозиторием и получить навык совместной работы над проектом.

В соответствии с заданием требуется настроить Git с использованием SSH-авторизации, обеспечивающей безопасное подключение к удаленному репозиторию без ввода пароля при каждой операции. Также необходимо выполнить fork исходного репозитория с проектом, чтобы получить собственную копию для дальнейшей работы. В рамках выполнения задания создаются две ветки – «lab1» и «ProjectTemplate», в которых настраивается файл «.gitignore», выполняются commit и push изменений, а также слияние веток в мастер после проверки преподавателем.

Для выполнения работы поставлены следующие задачи:

- 1 Установить и настроить Git (имя, группа, SSH-ключ).
- 2 Сделать fork репозитория и клонировать его локально.
- 3 Создать ветку «lab1», перенести материалы лабораторной работы №1, настроить «.gitignore», выполнить commit и push.
- 4 Создать ветку «ProjectTemplate», добавить README.md и пустой проект, настроить «.gitignore», выполнить commit и push.
- 5 Слить ветку «ProjectTemplate» в мастер после одобрения преподавателя.
- 6 Проанализировать изменения в репозитории на GitHub и оформить отчет.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Системы контроля версий

Система контроля версий (VCS) – программное обеспечение для отслеживания изменений файлов, сохранения истории и совместной работы над проектом. Основная задача VCS – фиксировать состояние проекта, чтобы можно было вернуться к предыдущим версиям и выявить, кем и когда была внесена модификация.

VCS делятся на централизованные (SVN, CVS) – с единственным центральным репозиторием, и распределенные (Git, Mercurial) – где каждый разработчик имеет полную локальную копию репозитория с историей.

Распределенный подход обеспечивает автономную работу и синхронизацию с другими репозиториями только при необходимости.

2.2 Git и его особенности

Git – распределенная система контроля версий, созданная Линусом Торвальдсом. Git стал стандартом в современной разработке благодаря гибкости, скорости и поддержке ветвления.

Ключевые особенности Git:

- локальность операций: почти все действия (коммит, история, ветки, слияние) выполняются без сетевого подключения;
- снимки, а не различия: Git сохраняет полный снимок файлов на момент коммита, а для не изменившихся – ссылку на предыдущую версию;
- легкое ветвление: ветки это указатели на коммиты, поэтому создание и слияние выполняются быстро;
- три состояния файлов: измененный (modified), подготовленный (staged) и зафиксированный (committed);
- область подготовки (staging area): позволяет выбирать, какие изменения войдут в следующий коммит.

2.3 Основные приемы работы с Git

Основной цикл работы с Git включает следующие этапы:

- 1 Клонирование удаленного репозитория создает локальную копию проекта.
- 2 Внесение изменений в файлы рабочего дерева.
- 3 Добавление изменений в область подготовки.
- 4 Фиксация подготовленных изменений в локальной истории.
- 5 Отправка изменений в удаленный репозиторий.

Для работы с ветками предусмотрены операции создания, переключения и слияния.

Файл «.gitignore» задает файлы и папки, игнорируемые Git (временные файлы, логи, артефакты сборки).

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Установка Git

Git был загружен с официального сайта git-scm.com, представленного на рисунке 3.1.1. На странице загрузки выбрана вкладка Windows и скачан автономный установщик последней версии для 64-битной системы.

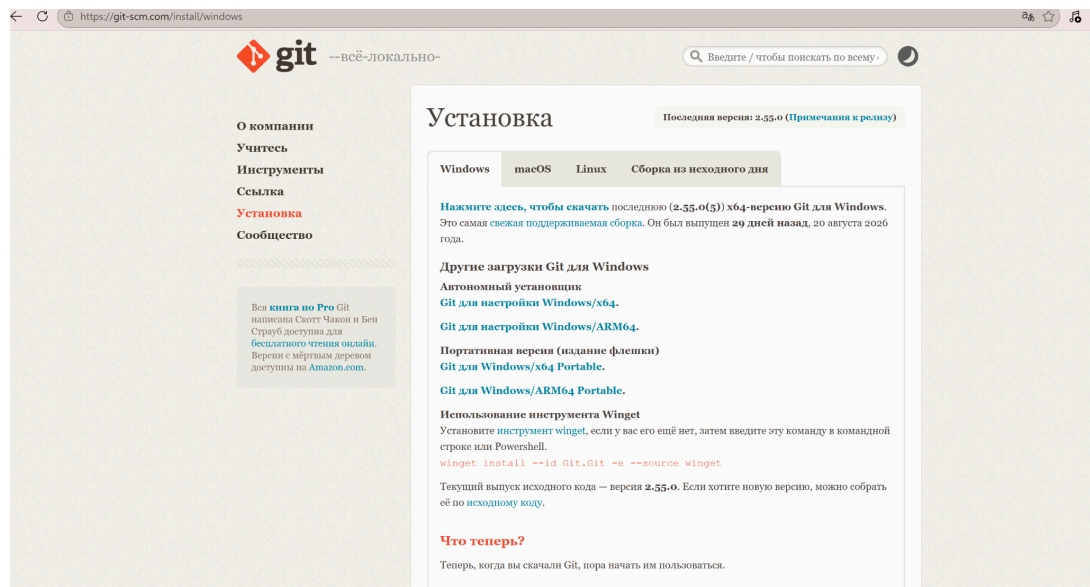


Рисунок 3.1.1 – Страница загрузки Git с официального сайта

Запущен скачанный установщик и выполнена установка с настройками по умолчанию, показанная на рисунке 3.1.2.

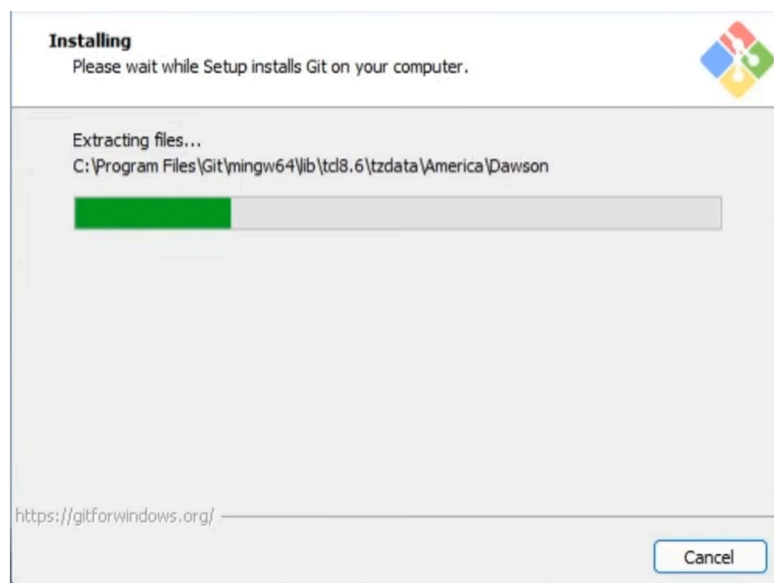
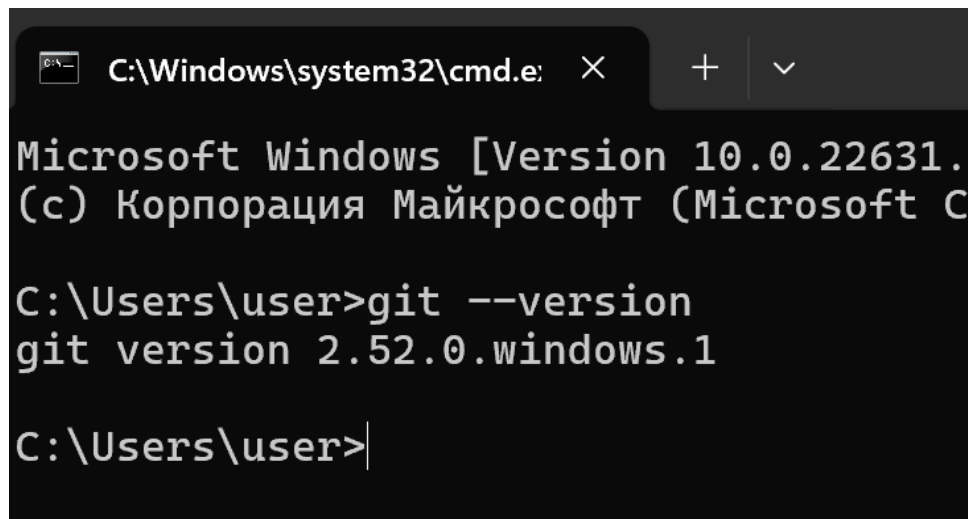


Рисунок 3.1.2 – Процесс установки Git

После завершения установки проверена корректность установки с помощью команды `git --version` в командной строке. Команда, представленная на рисунке 3.1.3, вывела версию Git, что подтверждает успешную установку.



```
C:\Windows\system32\cmd.e: X + v

Microsoft Windows [Version 10.0.22631.0]
(c) Корпорация Майкрософт (Microsoft Corporation)

C:\Users\user>git --version
git version 2.52.0.windows.1

C:\Users\user>|
```

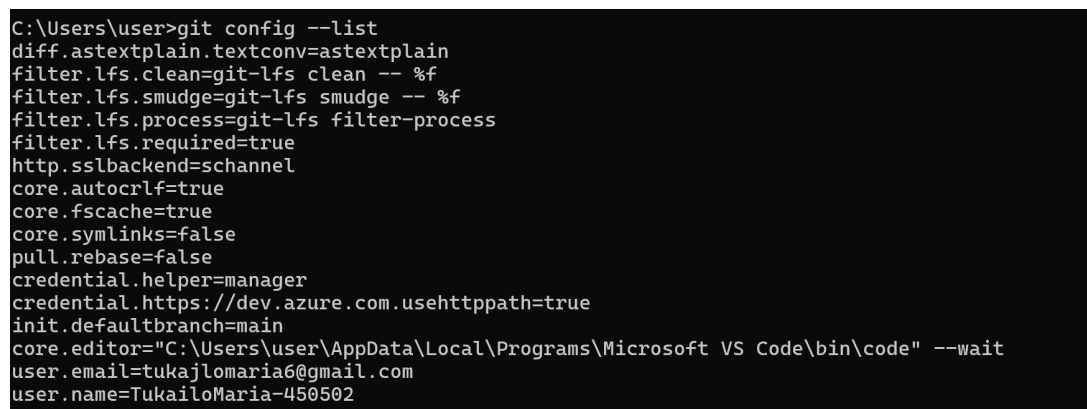
Рисунок 3.1.3 – Проверка версии Git

3.2 Настройка конфигурации Git

Для настройки Git были заданы имя пользователя и адрес электронной почты, которые будут отображаться в коммитах. Настройка выполнена с помощью следующих команд:

```
git config --global user.name "TukailoMaria-450502"
git config --global user.email "tukajlomaria6@gmail.com"
```

Проверка текущей конфигурации, представленная на рисунке 3.2.1, выполнена командой `git config --list`. В выводе команды отображаются заданные имя пользователя и email.



```
C:\Users\user>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main
core.editor="C:\Users\user\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
user.email=tukajlomaria6@gmail.com
user.name=TukailoMaria-450502
```

Рисунок 3.2.1 – Проверка конфигурации Git

В соответствии с заданием авторизация при работе с удаленным репозиторием выполняется только с использованием SSH-ключа. Для этого был сгенерирован SSH-ключ с помощью команды:

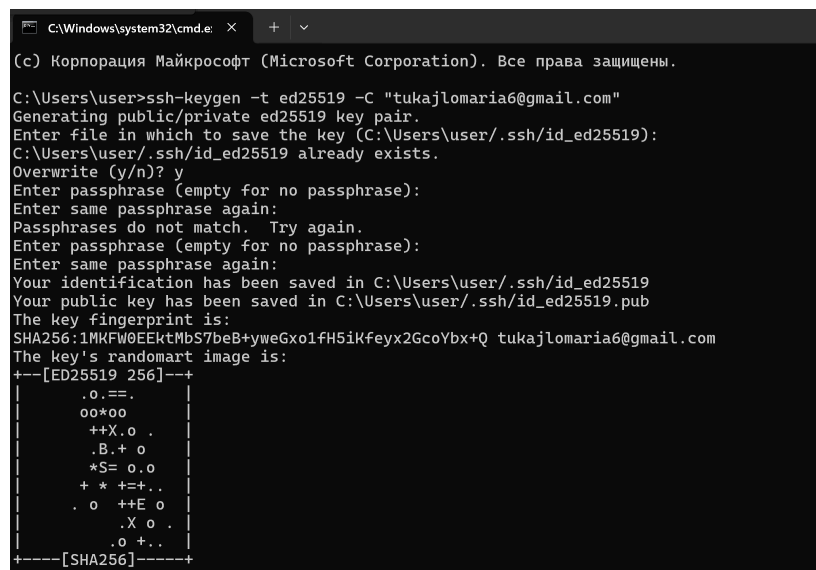
```
ssh-keygen -t ed25519 -C "tukajlomaria6@gmail.com"
```

При генерации ключа файл для сохранения ключа оставлен по умолчанию (C:\Users\user\.ssh\id_ed25519), на предложение перезаписать существующий файл дан ответ у, парольная фраза оставлена пустой.

В результате были созданы два файла:

- id_ed25519 – приватный ключ;
- id_ed25519.pub – публичный ключ.

Система вывела отпечаток ключа (SHA256) и его случайное графическое изображение, представленное на рисунке 3.2.2.



```
C:\Windows\system32\cmd.exe
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\user>ssh-keygen -t ed25519 -C "tukajlomaria6@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\user\.ssh\id_ed25519):
C:\Users\user\.ssh\id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Passphrases do not match. Try again.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\user\.ssh\id_ed25519
Your public key has been saved in C:\Users\user\.ssh\id_ed25519.pub
The key fingerprint is:
SHA256:1MKFW0EEktMbS7beB+yweGxo1fH5iKfeyx2GcoYbx+Q tukajlomaria6@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|  .o==. |
| oo*oo |
| ++X.o . |
| .B.+ o |
| *S= o.o |
| + * +=+. |
| . o ++E o |
| .X o . |
| .o +. |
+-----[SHA256]-----+
```

Рисунок 3.2.2 – Генерация SSH-ключа

Публичная часть ключа была скопирована и добавлена в настройки аккаунта GitHub в разделе New SSH key. Добавленный SSH-ключ на GitHub представлен на рисунке 3.2.3.

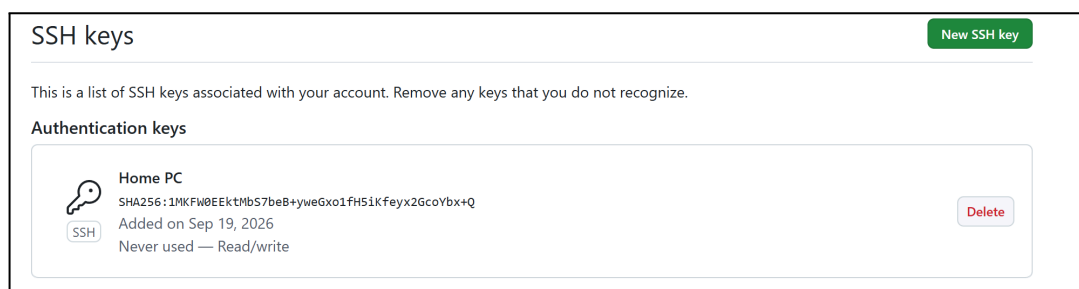


Рисунок 3.2.3 – SSH-ключ на GitHub

3.3 Fork репозитория

В соответствии с заданием был выполнен fork репозитория с проектом: BSUIR-SDLC/SDLC. Для этого на странице репозитория на GitHub нажата кнопка Fork, в результате чего в аккаунте MariaTukailo создана личная копия репозитория – MariaTukailo/SDLC, представленная на рисунке 3.3.1.

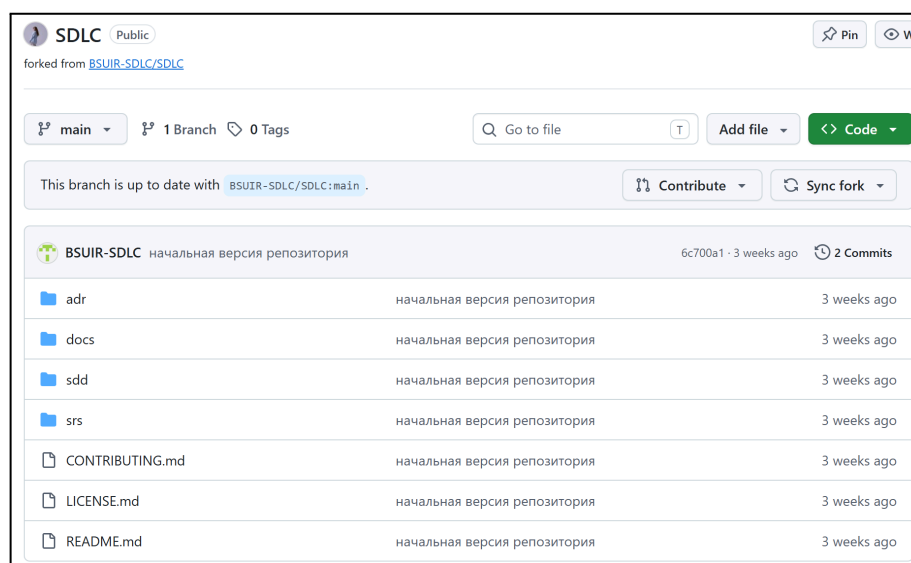


Рисунок 3.3.1 – Fork репозитория

Полученный репозиторий является публичным (отметка Public на странице репозитория), что обеспечивает возможность контролировать ход выполнения работы другим пользователям. На странице отображается информация об исходном репозитории, текущей ветке main, количестве веток и коммитов.

3.4 Выполнение операций с репозиторием в консоли

Работа с Git выполнялась только через командную строку. Локальная копия репозитория создана командой:

```
git clone git@github.com:MariaTukailo/SDLC.git
```

Клонирование выполнено по SSH, получено 30 объектов. Изучена структура репозитория.

Относительно текущего мастера создана новая ветка lab1 и выполнен переход в нее:

```
git checkout -b lab1
```

В локальную копию репозитория скопированы материалы лабораторной работы №1 в папку lab1/fate.

Изменения в рабочем дереве проанализированы командой:

```
git status
```

В разделе Untracked files отображалась новая папка lab1. Изменения в рабочем дереве представлены на рисунке 3.4.1.

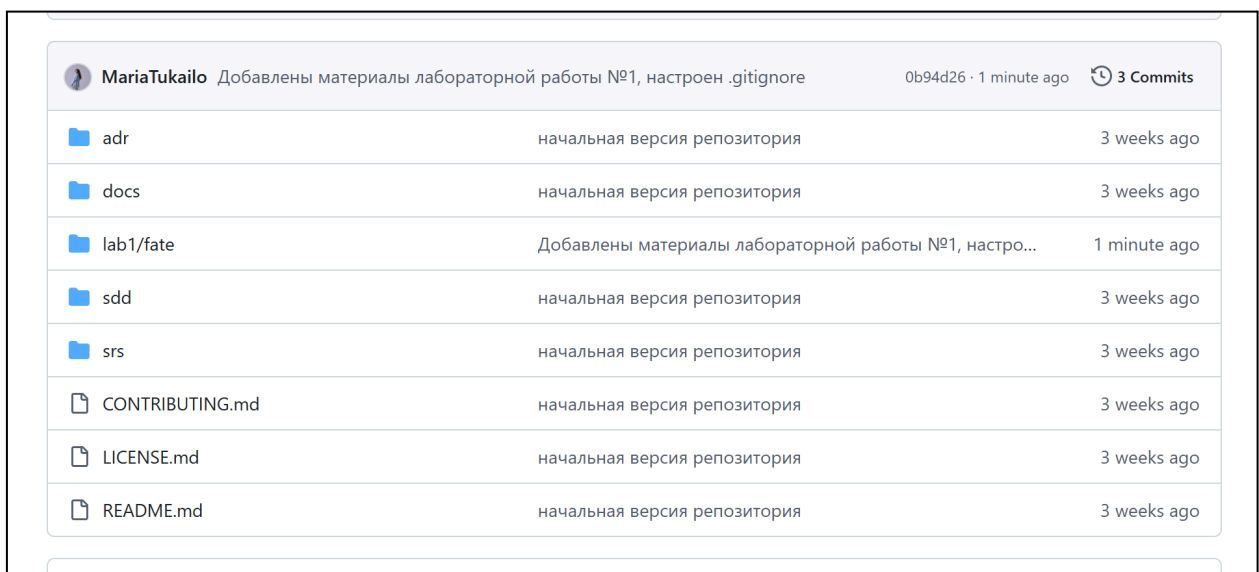
```
C:\Users\user\Desktop\SDLC>git status
On branch lab1
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    lab1/

nothing added to commit but untracked files present (use "git add" to track)
```

Рисунок 3.4.1 – Изменения в рабочем дереве

В корне проекта создан файл .gitignore, исключающий временные папки и файлы, не влияющие на сборку проекта. Содержимое файла .gitignore находится в приложении А.

Изменения добавлены в область подготовки, зафиксированы и отправлены в удаленный репозиторий, представленный на рисунке 3.4.2.



File/Folder	Description	Time
adr	начальная версия репозитория	3 weeks ago
docs	начальная версия репозитория	3 weeks ago
lab1/fate	Добавлены материалы лабораторной работы №1, настро...	1 minute ago
sdd	начальная версия репозитория	3 weeks ago
srs	начальная версия репозитория	3 weeks ago
CONTRIBUTING.md	начальная версия репозитория	3 weeks ago
LICENSE.md	начальная версия репозитория	3 weeks ago
README.md	начальная версия репозитория	3 weeks ago

Рисунок 3.4.2 – Ветка lab1 в удаленном репозитории

3.5 Создание ветки ProjectTemplate

После завершения работы в ветке lab1 выполнен возврат в мастер:

```
git checkout main
```

Относительно текущего мастера создана новая ветка ProjectTemplate и выполнен переход в нее:

```
git checkout -b ProjectTemplate
```

В корневой папке репозитория создан файл README.md, содержащий информацию о проекте: автора, название дисциплины, описание создаваемого проекта, используемые технологии и структуру репозитория, представленный в приложении Б.

В репозитории создан пустой Java-проект app на базе JDK 17.

В корне проекта настроен файл .gitignore, представленный в приложении В, исключающий временные папки и файлы, не влияющие на сборку проекта.

Изменения добавлены в область подготовки, зафиксированы и отправлены в удаленный репозиторий, представленный на рисунке 3.5.1.

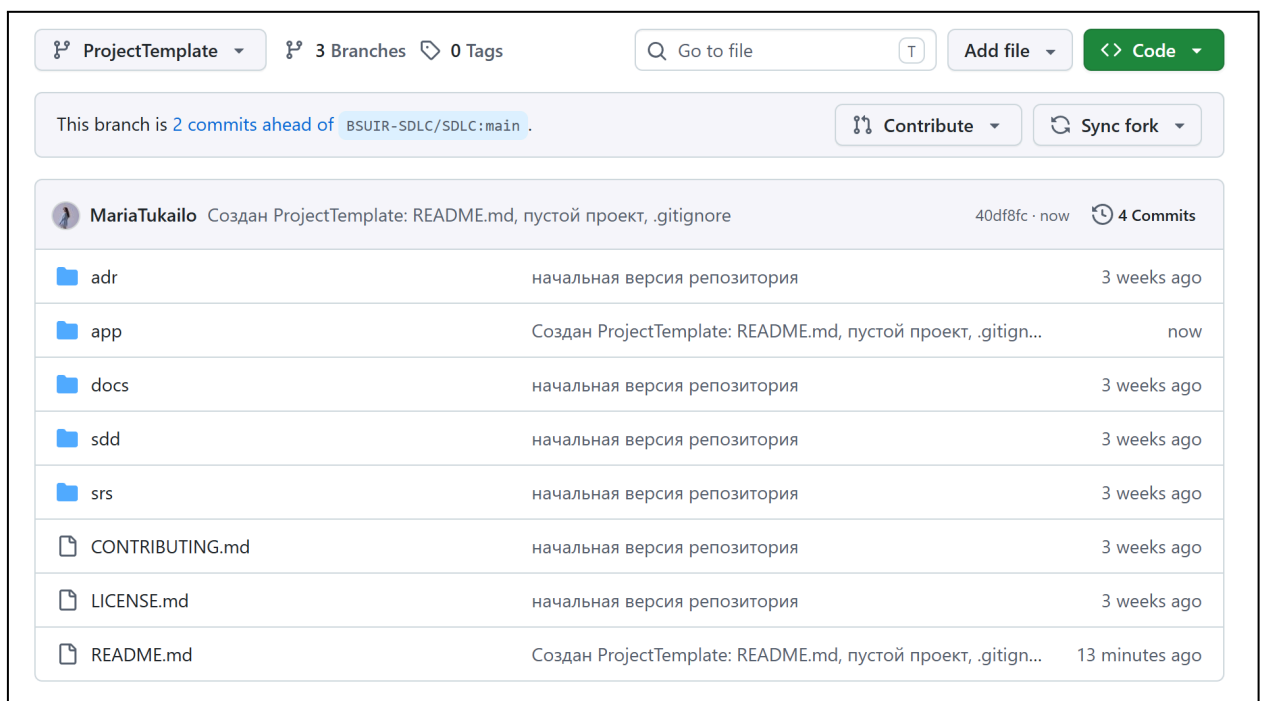


Рисунок 3.5.1 – Ветка ProjectTemplate в удаленном репозитории

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 В чем преимущество Git перед SVN?

Git является распределенной системой контроля версий, тогда как SVN – централизованной. Главное преимущество Git заключается в том, что практически все операции выполняются локально: коммит, просмотр истории, создание веток и слияние не требуют подключения к серверу. Кроме того, Git обеспечивает легкое и быстрое ветвление, высокую скорость работы и полную локальную копию репозитория с историей, что делает его устойчивым к сбоям центрального сервера.

4.2 В чем преимущество распределенных репозиториях?

В распределенных системах каждый разработчик имеет полную локальную копию репозитория, включая всю историю изменений. Это позволяет работать автономно, без подключения к сети, и синхронизироваться с другими репозиториями только при необходимости. Кроме того, распределенные репозитории обеспечивают устойчивость к потере центрального сервера и поддерживают гибкие модели совместной работы.

4.3 Граф типичной работы с веткой master в Git.

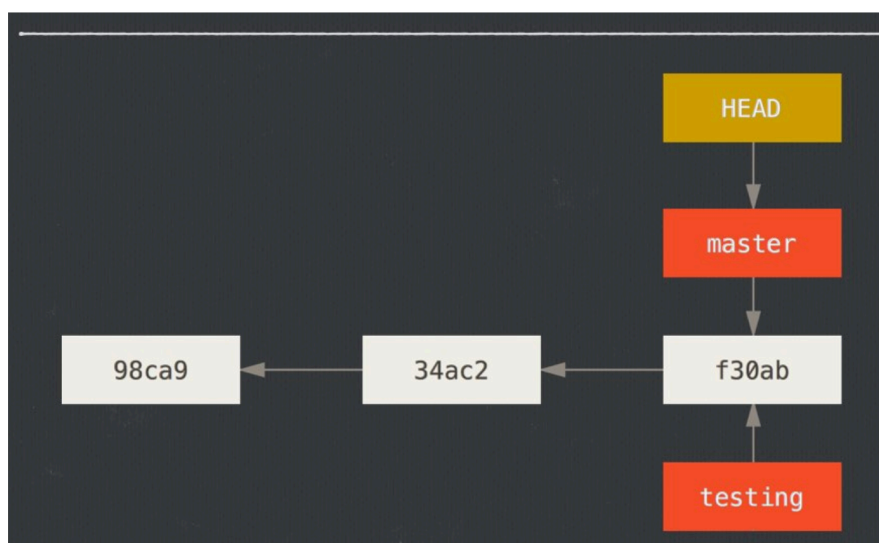


Рисунок 4.3.1 – граф работы с веткой master

4.4 Наиболее часто используемые команды командной строки.

К наиболее часто используемым командам Git относятся: `git clone` – для клонирования удаленного репозитория, `git status` – для просмотра состояния рабочего дерева, `git add` – для добавления изменений в область

подготовки, `git commit` – для фиксации изменений в локальной истории, `git push` – для отправки изменений в удаленный репозиторий, `git pull` и `git fetch` – для получения изменений из удаленного репозитория, `git branch`, `git checkout` и `git merge` – для работы с ветками, а также `git log` – для просмотра истории коммитов.

4.5 Что нужно делать, если кто-то успел внести в удаленный репозиторий изменения?

Если кто-то уже успел внести изменения в удаленный репозиторий, перед отправкой своих изменений необходимо сначала получить эти изменения к себе командой `git pull`.

Эта команда выполняет два действия: загружает изменения из удаленного репозитория и сразу выполняет их слияние с текущей локальной веткой. Если изменения не конфликтуют, Git автоматически выполнит слияние, после чего можно отправить свои изменения командой `git push`.

Если же возник конфликт, Git сообщит о нем – потребуется вручную разрешить конфликтующие файлы, затем добавить их в область подготовки командой `git add`, зафиксировать командой `git commit` и отправить в удаленный репозиторий командой `git push`. В качестве альтернативы можно использовать `git fetch` с последующим `git merge` или `git pull --rebase` для получения линейной истории без лишнего коммита слияния.

Такой подход позволяет избежать потери чужих изменений, которые уже были зафиксированы другими участниками проекта, и корректно синхронизировать локальную и удаленную версии репозитория, обеспечивая согласованность истории коммитов и целостность проекта при совместной работе.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была изучена система контроля версий Git: рассмотрены ее особенности, преимущества перед централизованными системами, а также основные приемы работы с локальным и удаленным репозиторием. Выполнена установка Git и его первоначальная настройка с указанием имени пользователя, номера группы и адреса электронной почты, а также настроена авторизация с использованием SSH-ключа.

Для выполнения задания был выполнен fork предложенного репозитория с проектом, после чего создана его локальная копия с помощью операции clone. Изучена структура и наполнение папок репозитория. Относительно текущего мастера создана ветка lab1, в которую скопированы материалы лабораторной работы №1, проанализированы изменения в рабочем дереве, настроен файл .gitignore для исключения временных папок и файлов, не влияющих на сборку проекта, выполнены commit и push изменений в удаленный репозиторий.

Далее выполнен возврат в мастер и создана ветка ProjectTemplate, в которой сформирован файл README.md с описанием проекта, создан пустой проект для дальнейшего проектирования и разработки, а также настроен файл .gitignore. Изменения зафиксированы и отправлены в удаленный репозиторий.

Все действия выполнялись исключительно через командную строку, без использования графических интерфейсов для работы с Git, что позволило закрепить практические навыки работы с основными командами.

Таким образом, все задачи, поставленные в рамках лабораторной работы, были выполнены: изучена система контроля версий Git, освоены основные операции с репозиторием, получен навык совместной работы над проектом с использованием ветвления, слияния и SSH-авторизации, а также оформлен репозиторий в соответствии с требованиями к структуре проекта.

ПРИЛОЖЕНИЕ А
(обязательное)
Содержимое файла .gitignore проекта fate

.idea/
*.iml
out/
!**/src/main/**/out/
!**/src/test/**/out/

build/
target/
bin/
obj/

*.tmp
*.log
*.bak
Thumbs.db
desktop.ini

.DS_Store

ПРИЛОЖЕНИЕ Б
(обязательное)
Содержимое файла README.md

Проект SDLC

****Автор:**** Тукайло М.А., гр. 450502

****Дисциплина:**** Жизненный цикл разработки программного обеспечения

****Описание проекта:**** В данном репозитории будет выполняться проектирование и разработка задания по дисциплине.

ПРИЛОЖЕНИЕ В
(обязательное)
Содержимое файла .gitignore проекта app

.idea/
*.iml
out/
!**/src/main/**/out/
!**/src/test/**/out/

build/
target/
bin/
obj/

*.tmp
*.log
*.bak
Thumbs.db
desktop.ini

.DS_Store